Algoritmo de Grover

Genesis Silesky

20 de octubre de 2026

1. Algoritmo de Grover: búsqueda cuadrática

1.1. Objetivo

Implementar Grover para $N = 8$ elementos y comparar el número de consultas con la búsqueda clásica, dejando claro que la ventaja es *cuadrática*, no exponencial.

1.2. Conceptos previos

Concepto previo: oráculo, reflexión y complejidad de consultas

- **Oráculo:** una caja negra O que reconoce la solución. Aquí $O|x\rangle = -|x\rangle$ si x es el elemento marcado y $O|x\rangle = |x\rangle$ si no.
- **Complejidad de consultas:** se cuenta cuántas veces se usa el oráculo, no el tiempo total. Clásico: $O(N)$; Grover: $O(\sqrt{N})$.
- **Interferencia** (capítulo 05a): Grover amplifica la amplitud de la respuesta correcta.

1.3. Fundamento teórico

Se parte de la superposición uniforme $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_x |x\rangle$ con $N = 2^n$. Cada iteración aplica $G = D\,O$, donde $D = 2|s\rangle\langle s| - I$ ("difusión": reflexión respecto al promedio de las amplitudes). Geométricamente G gira el estado, en el plano generado por $|s\rangle$ y la solución $|w\rangle$, un ángulo 2θ con

$$\sin\theta = \frac{1}{\sqrt{N}}. \tag{1}$$

Tras k iteraciones, la probabilidad de medir la solución es

$$P_k = \sin^2\big((2k+1)\theta\big), \qquad k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{N}. \tag{2}$$

Para $N = 8$: $\theta = \arcsin(1/\sqrt{8}) \approx 0.3614$ y $k = \lfloor \frac{\pi}{4}\sqrt{8} \rfloor = 2$, de donde $P_2 = \sin^2(5\theta) \approx 0.945$.

Contraste. Clásicamente, buscar un elemento entre N sin estructura requiere $\sim N/2$ consultas en promedio; Grover usa $\sim \frac{\pi}{4}\sqrt{N}$ (mejora cuadrática, que se sabe óptima para este problema). Esto es muy distinto de Shor (factorización), cuya ventaja sobre los mejores algoritmos clásicos conocidos es *exponencial*.

Oráculo en el código. Para marcar $|101\rangle$ se aplica X a los qubits donde esa cadena tiene un 0 (convirtiéndola en $|111\rangle$), luego un CCZ (hecho como H–Toffoli–H en el último qubit), y se deshacen las X. El CCZ solo cambia el signo cuando los tres qubits valen 1.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 06 - Grover: busqueda cuadratica O(sqrt(N)) frente a O(N)

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
# grover_operator arma el operador de Grover G = (difusion)(oraculo).
from qiskit.circuit.library import grover_operator
from qiskit.quantum_info import Statevector

# %% Celda 2: el oraculo
# Problema: entre N = 2^3 = 8 cadenas, encontrar la marcada "101".
# El oraculo es una puerta que cambia el signo (fase -1) SOLO de la
# cadena marcada, sin revelar cual es: U|x> = -|x> si x == "101", y |x> si no.
n, marcado = 3, "101"

# Qiskit numera el qubit 0 a la derecha de la cadena, por eso se invierte
# con reversed(). Buscamos los qubits donde la cadena marcada tiene un 0.
ceros = [i for i, b in enumerate(reversed(marcado)) if b == "0"]

oraculo = QuantumCircuit(n)
oraculo.x(ceros)                      # convierte "101" en "111" temporalmente
oraculo.h(n - 1)                      # H - mcx - H sobre el ultimo qubit
oraculo.mcx(list(range(n - 1)), n - 1)  # equivale a un CCZ: fase -1 si todo es 1
oraculo.h(n - 1)
oraculo.x(ceros)                      # deshace el primer paso

# %% Celda 3: aplicar Grover
G = grover_operator(oraculo)          # un paso = oraculo + difusion
# El numero optimo de iteraciones es ~ (pi/4)*sqrt(N). Aqui es 2.
iteraciones = int(np.floor(np.pi / 4 * np.sqrt(2 ** n)))

qc = QuantumCircuit(n)
qc.h(range(n))                        # superposicion uniforme de las 8 cadenas
for _ in range(iteraciones):
    qc.compose(G, inplace=True)       # agrega G al final del circuito
print("Iteraciones de Grover:", iteraciones)

# Cada iteracion amplifica la amplitud de la cadena marcada por interferencia.
probs = Statevector(qc).probabilities_dict()
print("P(", marcado, ") =", round(float(probs[marcado]), 3))  # ~0.945

# %% Celda 4: comparacion de consultas
# Clasico: en promedio hay que revisar N/2 elementos. Grover: ~(pi/4)*sqrt(N).
# La ventaja es cuadratica (no exponencial como en Shor).
for N in [8, 1_000, 1_000_000, 10**12]:
    print(f"N={N:>16,}  clásico ~ {N // 2:>16,}  Grover ~ {int(np.pi / 4 * np.sqrt(N)):>10,}")
```

1.5. Explicación celda por celda

Celda 2 Construye el oráculo. `reversed(marcado)` se usa porque Qiskit numera el qubit 0 a la derecha. `mcx` es una puerta X con varios controles.

Celda 3 `grover_operator` arma G a partir del oráculo. Se calcula k_{opt}, se aplica H a todos y se repite G k veces. `Statevector` da la probabilidad exacta de `"101"`.

Celda 4 Tabla comparativa de consultas: la ventaja crece con N pero solo como raíz.

1.6. Salida obtenida

```
Iteraciones de Grover: 2
P( 101 ) = 0.945
N=               8  clásico ~               4  Grover ~          2
N=           1,000  clásico ~             500  Grover ~         24
N=       1,000,000  clásico ~         500,000  Grover ~        785
N=1,000,000,000,000  clásico ~  500,000,000,000  Grover ~    785,398
```

Coincide con la teoría: $P_2 \approx 0.945$.

1.7. Interpretación

Con $N = 10^{12}$ Grover necesita unas 7.9×10^5 consultas frente a 5×10^{11}: una mejora enorme, pero cuadrática. Además cada consulta cuántica puede ser mucho más cara que una clásica, y el oráculo debe construirse como un circuito; por eso la ventaja práctica depende del problema y del hardware.

1.8. Ejercicios

1. Cambie `marcado` a `"010"` y verifique el resultado.
2. Use 1 y 3 iteraciones en lugar de 2: ¿qué pasa con P_k? (Pista: es una función periódica de k; "pasarse" empeora el resultado.)
3. Extienda a $n = 4$ y calcule el k_{opt}.